


UNIVERSIDADE FEDERAL DO MARANHÃO
CENTRO DE CIÊNCIAS EXATAS E TECNOLOGIA

# **RELATÓRIO DE EXECUÇÃO:** DESENVOLVIMENTO DE UM SIMULADOR DE MEMÓRIA RAM COM OPERAÇÕES DE LEITURA E ESCRITA EM C.

Alunos: Caio Ferreira Sousa, Danilo Nava Soares Lima, Raynara Ribeiro Alves , Rhuã Yuri Nascimento Sardinha, Yuri Aguiar Gonçalves

São Luís - MA
2026

## 1. INTRODUÇÃO

A memória RAM é um dos principais componentes da arquitetura de computadores, sendo responsável pelo armazenamento temporário de dados e informações utilizadas durante a execução de programas. O acesso à memória é realizado por meio de endereços, permitindo que o processador identifique a posição na qual determinado dado está armazenado.

Com o objetivo de facilitar a compreensão desses conceitos, este projeto propõe o desenvolvimento de um simulador de memória RAM em linguagem C, capaz de representar, de maneira simplificada, operações fundamentais de uma memória, como escrita, leitura, sobrescrita e consulta dos dados armazenados.

O simulador será executado por meio de uma interface interativa no terminal, permitindo ao usuário informar um endereço de memória e um dado a ser armazenado, consultar valores previamente registrados e visualizar o estado da memória. Dessa forma, o projeto busca relacionar os conceitos teóricos estudados na disciplina de arquitetura de computadores com uma implementação prática desenvolvida em linguagem C.

## 2. JUSTIFICATIVA

A memória RAM exerce papel fundamental no funcionamento de um computador, pois possibilita o armazenamento temporário de informações necessárias para a execução dos programas. A compreensão de conceitos como endereçamento, leitura, escrita e armazenamento de dados é importante para o estudo da organização e arquitetura de computadores.

A implementação de um simulador em linguagem C permite representar esses conceitos de forma prática, proporcionando uma visão simplificada do funcionamento de uma memória. O uso da linguagem C também contribui para a compreensão de estruturas de dados, vetores, funções, ponteiros e gerenciamento de informações em baixo nível, conceitos diretamente relacionados à área de arquitetura de computadores.

## 3. CRONOGRAMA

O projeto está dividido em 3 etapas de trabalhos. Cada etapa possui um foco específico. A seguir o cronograma completo:

- **Semana 1:** Revisão sobre memória RAM, endereçamento, leitura, escrita e organização dos dados.
- **Semana 2:** Planejamento do programa e definição da estrutura da memória.

- **Semana 3:** Implementação da estrutura inicial da memória e da operação de escrita.
- **Semana 4:** Implementação da operação de leitura e tratamento de endereços inválidos ou vazios.
- **Semana 5:** Implementação da visualização completa do estado da memória.
- **Semana 6:** Desenvolvimento do menu interativo no terminal e integração de todas as operações.
- **Semana 7:** Testes completos: escrita, leitura, sobrescrita, endereço inválido e memória vazia.
- **Semana 8:** Organização do código, documentação, GitHub, relatório e apresentação.

## 3. GRÁFICO DE GANTT

A seguir um cronograma no formato gráfico de Gantt, de forma mais resumida dos passos a serem seguidos ao longo dos meses. Partindo do aprofundamento das pesquisas ao desenvolvimento do programa e sua implementação.




Para o planejamento e acompanhamento do cronograma, utilizou-se a plataforma ClickUp na construção do gráfico de gantt. A ferramenta permitiu a estruturação do projeto em etapas e subtarefas, o estabelecimento de prazos bem definidos e a atribuição de responsáveis, assegurando uma visualização clara do fluxo de trabalho e do progresso das entregas.

Com o intuito de mitigar riscos de atraso e otimizar a produtividade, a distribuição das tarefas priorizou a execução em duplas. Não obstante, a etapa de implementação da estrutura de memória registrou um desvio no cronograma planejado. Esse atraso pontual decorreu da necessidade de um aprofundamento nas discussões de arquitetura e design da solução, visando definir a melhor abordagem técnica e garantir a eficiência e estabilidade da estrutura antes da sua consolidação.

## 4. IMPLEMENTAÇÃO

O código implementa um simulador em C que modela o comportamento básico de uma memória RAM de 256 posições. A estrutura central do programa é definida pela struct MemoriaRAM, que abstrai o espaço de endereçamento utilizando vetores para armazenar os dados em bytes (uint8_t), controlar a ocupação das posições e contabilizar as operações realizadas. Através de rotinas dedicadas de validação e gerenciamento, o sistema garante que apenas endereços válidos (de 0 a 255) e valores no limite de 8 bits (de 0 a 255) sejam processados. A seguir as pode-se visualizar as funcionalidades do programa:

```
/*
 * ============================================================
 * SIMULADOR DE MEMORIA RAM
 * Disciplina: Arquitetura de Computadores
 *
 * Funcionalidades:
 * 1 - Escrever dado na memoria
 * 2 - Ler dado da memoria
 * 3 - Exibir memoria
 * 4 - Limpar memoria
 * 0 - Sair
 *
 * Memoria simulada: 256 enderecos
 * ============================================================
```

A dinâmica de funcionamento simula as operações fundamentais de barramento e armazenamento de uma memória real. Na rotina de escrita, o programa verifica se o endereço já possui dados, sobrescreve o valor se necessário e marca a posição como ocupada. Na operação de leitura, ele consulta o estado da célula antes de retornar o dado contido nela. Além disso, a aplicação mantém estatísticas de acesso e oferece funções para exibir o mapa de memória, listando apenas os endereços efetivamente utilizados, e para restaurar todo o espaço ao estado inicial zerado.

```
#define TAM_MEMORIA 256
#define TAM_DADO 8


typedef struct {
    uint8_t dados[TAM_MEMORIA];
    int ocupado[TAM_MEMORIA];
    int total_escritas;
    int total_leituras;
} MemoriaRAM;
```

A interação com o usuário é gerida por um menu em laço de repetição (do-while) operado via comandos de terminal. A implementação conta com tratamento de erros de

entrada de dados, utilizando validações no retorno da função scanf e a rotina limpar_buffer para evitar travamentos decorrentes de resíduos no buffer do teclado.